ReferrAI — Software Project Execution Plan

Version 1.0 · Companion to ROADMAP.md and TASK_BREAKDOWN.md Team: Vidyuth & Mukul · **Duration:** ~12 weeks (single semester, alongside coursework) **Status at plan start:** Planning, architecture, DB design, folder structure, and Git repo complete. Zero application code written.

> This document is an execution and coordination layer on top of the existing ROADMAP and TASK_BREAKDOWN. Where it references work, it uses the existing task IDs (e.g. `REF-003`, `AI-FRD-002`). The BRD and SRS remain the source of truth for *what* is built; this plan governs *who*, *when*, and *in what order*.

---

0. Guiding Strategy (read this first)

Three decisions shape everything below:

1. **Vertical ownership, not horizontal.** Neither developer is "the backend person" or "the frontend person." Each owns complete features end-to-end: models → repository → service → route → React page → tests → docs. This is how both gain full-stack experience and how merge conflicts stay low (you rarely touch the same file).

2. **The ML track runs in parallel from Week 3.** `ml/` depends only on the finalized DB schema, not on a running backend. Vidyuth develops synthetic generators and trains/evaluates models in `ml/` while Mukul builds the core platform. The hard, time-consuming, eval-gated ML work therefore starts early and finishes as integration, not invention.

3. **Mukul owns the financial core; Vidyuth owns the data + intelligence.** Mukul takes auth, commission, wallet, payouts, partner management, and the admin dashboard — the transaction-critical, atomicity-sensitive backend. Vidyuth takes referral capture (the data source for the models), settings, and the AI module. This is a natural seam: the things Vidyuth's models *consume* (clicks, conversions) are the things Vidyuth *produces*.

Workload balance (rough developer-hours, ~210h total incl. shared overhead):

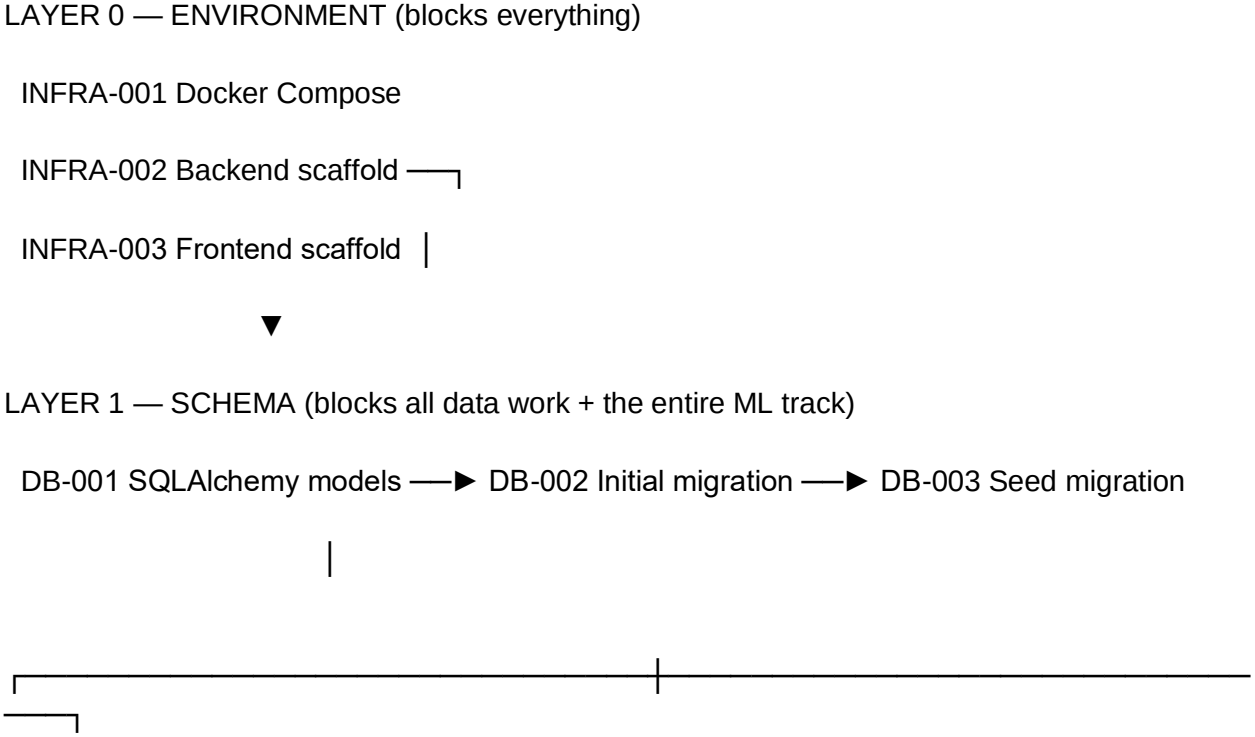
Developer	Foundation	Core Platform	Dashboards	AI Features	Hardening	Total
Vidyuth	~9h	~23h (Referral + Settings)	~6h (Partner side)	~56h (5 ML features + XAI core)	~6h	**~101h**
Mukul	~6h	~52h (Auth + Commission + Partner + Wallet)	~12h (Admin side)	~28h (Ranking + Fraud ops + Discovery API/UI)	~9h	**~109h**

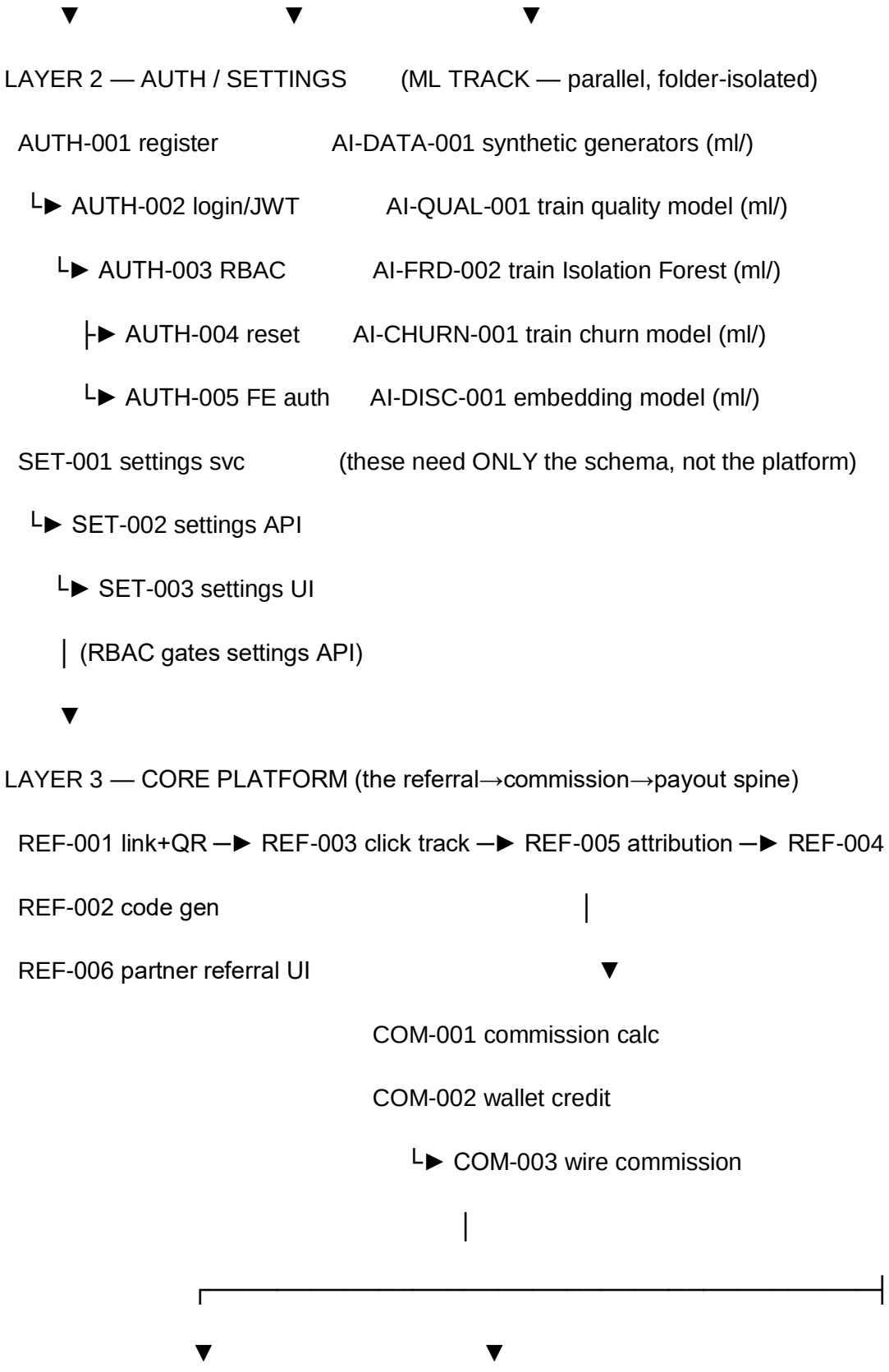
The hour counts are deliberately close. Vidyuth's are slightly lower because AI work carries higher *difficulty and risk per hour* (eval gates, model tuning, SHAP), so effective load is comparable. Treat these as planning numbers; rebalance at each weekly sprint check-in.

---

1. Feature Dependency Graph

Read top to bottom. Anything that depends on something above it cannot start until that thing is merged to `main`.




LAYER 2 — AUTH / SETTINGS (ML TRACK — parallel, folder-isolated)
AUTH-001 register
AI-DATA-001 synthetic generators (ml/)
AUTH-002 login/JWT
AI-QUAL-001 train quality model (ml/)
AUTH-003 RBAC
AI-FRD-002 train Isolation Forest (ml/)
AUTH-004 reset
AI-CHURN-001 train churn model (ml/)
AUTH-005 FE auth
AI-DISC-001 embedding model (ml/)
SET-001 settings svc
(these need ONLY the schema, not the platform)
SET-002 settings API
SET-003 settings UI
(RBAC gates settings API)
LAYER 3 — CORE PLATFORM (the referral→commission→payout spine)
REF-001 link+QR
REF-003 click track
REF-005 attribution
REF-004 conversion
REF-002 code gen
REF-006 partner referral UI
COM-001 commission calc
COM-002 wallet credit
COM-003 wire commission

PRT-003 tier upgrade WAL-001/002/003 withdrawal

(needs COM-002 + PRT-002) WAL-004 admin review

PRT-001 profile API PRT-002 tier CRUD WAL-005/006 UIs

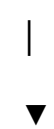
PRT-004/005 partner UIs



LAYER 4 — DASHBOARDS (needs core platform complete)

RPT-001 admin API ─► RPT-003 CSV ─► RPT-004 admin UI

RPT-002 partner API ─► RPT-005 partner UI



LAYER 5 — AI INTEGRATION (needs platform + trained models from ML track)

AI-DATA-002 synthetic API

AI-QUAL-002 quality svc ─► AI-QUAL-003 wire into conversion ─► AI-QUAL-004 UI

AI-FRD-001 rules ─► AI-FRD-003 fraud svc ─► AI-FRD-004 wire into click

├► AI-FRD-005 review API ─► AI-FRD-007 UI

└► AI-FRD-006 withdrawal gate (touches wallet)

AI-CHURN-002 svc ─► AI-CHURN-003 scheduled ─► AI-CHURN-004 UI

AI-RANK-001 (needs QUAL-002 + CHURN-002) ─► AI-RANK-002 API ─► AI-RANK-003 UI

AI-DISC-002 ingestion API ─► AI-DISC-003 UI

AI-XAI-001 SHAP (needs all 3 models) ─► AI-XAI-002 attach ─► AI-XAI-003 UI

▼

LAYER 6 — HARDENING (needs everything)

TST-001 integration · TST-002 degradation · TST-003 perf · DEMO-001/002

Per-feature dependency summary

Feature	**Prerequisites (must be merged first)**	**Blocks (cannot start until this is done)**	**Parallelizable with**
Foundation/Infra	—	Everything	Frontend scaffold ‖ backend scaffold
Schema (models/migrations)	Backend scaffold	Auth, Settings, Referral, **entire ML track**	—
Auth + RBAC	Schema	Settings API, all protected routes	ML track, Settings service
Settings	Schema, RBAC (for API)	Commission, Attribution, Fraud/Churn thresholds	Auth, Referral link/code
Referral capture	Schema, RBAC	Commission, Quality scoring, Fraud scoring	Commission service (until wiring), ML track
Commission	Settings, Schema	Wallet credit, Tier upgrade, Ranking	Partner profile/tier CRUD
Partner mgmt	RBAC, Commission (for tier upgrade)	Ranking, Discovery similarity	Wallet, Referral UI
Wallet/Payout	Commission (credit svc)	Fraud withdrawal gate	Dashboards, Partner UI
Dashboards	Core platform complete	— (consumes everything)	ML track tail end
Synthetic data	Schema	All model training	Core platform (different folder)

Feature	Prerequisites (must be merged first)	Blocks (cannot start until this is done)	Parallelizable with
Quality / Fraud / Churn models	Synthetic data	Their scoring services, XAI, Ranking	Each other (independent models)
Ranking	Quality svc + Churn svc + Settings	—	Discovery
Discovery	Schema, Partner profiles	—	Ranking, XAI
XAI	All 3 trained models	— (cross-cuts all score displays)	Discovery

---

2. Project Timeline (12 weeks)

Assumes ~8–9 productive hours/developer/week, weekday time scarce, weekends heavier. **Week 7 is deliberately low-velocity for internal assessments.** Weeks 11–12 are buffer + demo.

Notation: **[V]** = Vidyuth, **[M]** = Mukul, **[V‖ M]** = parallel independent work, **[V+M]** = collaborative/pair.

Week 1 — Foundation Pt 1: Environment + Schema

- [M] `INFRA-001` Docker Compose (db, backend, frontend, nginx). [M] `INFRA-003` frontend scaffold (Vite, Router, Axios+JWT interceptor, layout shells).
- [V] `INFRA-002` backend scaffold (config, async engine/session, requirements). [V] `DB-001` all SQLAlchemy 2.0 models (lead), `DB-002` initial migration.
- **[V+M] Schema sign-off meeting** — both review every table/constraint/index against DATABASE_DESIGN.md before the migration is locked. *This is the highest-leverage collaboration point in the project.*
- **Deliverable:** `docker-compose up` brings the stack up; `alembic upgrade head` creates all tables.
- **Milestone / tag:** `v0.1-foundation`

Week 2 — Foundation Pt 2: Auth + Settings

- [M] `AUTH-001` register → `AUTH-002` login/JWT → `AUTH-003` RBAC deps → `AUTH-004` reset → `AUTH-005` frontend auth flow.

- [V] `DB-003` seed migration (default tier + canonical settings) → `SET-001` settings service/repo → `SET-002` settings API → `SET-003` settings admin UI.
- **[V+M] API contract:** agree request/response shapes for auth + settings (write them down in `docs/` before coding the frontend).
- **Exit (Phase 1):** admin logs in, edits a setting via UI, an `audit_logs` row appears.
- **Milestone:** `v0.2-auth-settings` — **end of Phase 1**

Week 3 — Core Pt 1 + **ML track kickoff**

- [V] `REF-001` link + QR, `REF-002` code gen, `REF-003` click tracking endpoint.
- [M] `COM-001` commission calc service, `COM-002` wallet credit service (`WalletService.credit` + Transaction).
- [V‖ evenings/weekend] **ML track begins:** `AI-DATA-001` synthetic generators in `ml/generators/` (writes labelled datasets to `ml/data/synthetic/`). Needs only the schema.
- **[V+M] Contract:** the conversion→commission handoff (what `CommissionService.calculate_for_conversion` receives).

Week 4 — Core Pt 2: Conversion spine closes

- [V] `REF-005` attribution service, `REF-004` conversion endpoint, `REF-006` partner referral UI.
- [M] `COM-003` wire commission into conversion, `PRT-001` partner profile API, `PRT-002` tier CRUD.
- **Integration checkpoint #1:** click → conversion → commission → wallet credit works end-to-end via API.
- **Milestone:** `v0.3-referral-commission`

Week 5 — Core Pt 3: Tier upgrade + Wallet/Payout

- [M] `PRT-003` automatic tier upgrade, `WAL-001` wallet API, `WAL-002` withdrawal request, `WAL-003` debit, `WAL-004` admin review API. **Mukul owns all of `wallet_service.py`** (credit + debit) to prevent conflicts.
- [V] `PRT-005` partner profile UI (tier progress bar), continue ML track: finish generators, start `AI-QUAL-001` quality model training in `ml/training/`.
- **Milestone:** `v0.4-wallet-payout`

Week 6 — Phase 2 close + Dashboards start

- [M] `WAL-005` wallet UI, `WAL-006` admin payout UI, begin `RPT-001` admin dashboard API.

- [V] `PRT-004` admin partner list/detail UI (score cards as placeholders), `RPT-002` partner dashboard API; ML track: train fraud (`AI-FRD-002`) Isolation Forest.
- **Integration checkpoint #2 (Phase 2 exit):** partner generates link → simulates click → records conversion → sees commission in wallet → requests withdrawal (auto-approved below threshold, queued above). Tier upgrade fires at threshold.
- **Milestone:** `v0.5-core-platform` — **end of Phase 2**

Week 7 — INTERNAL ASSESSMENTS (low velocity) + finish Dashboards

- Reduced development. Protect this week.
- [M] `RPT-003` CSV export, `RPT-004` admin dashboard UI (KPI cards, volume chart).
- [V] `RPT-005` partner dashboard UI; light ML: train churn model (`AI-CHURN-001`), evaluate all models against SRS targets (ROC-AUC ≥ 0.75, churn PR-AUC).
- **Milestone:** `v0.6-dashboards` — **end of Phase 3**

Week 8 — AI Pt 1: Synthetic + Quality (integration)

- [V] `AI-DATA-002` synthetic data API (wire generators into backend, bulk insert tagged `is_synthetic`), `AI-QUAL-002` quality scoring service, `AI-QUAL-003` wire into conversion `BackgroundTask`, `AI-QUAL-004` UI.
- [M] `AI-FRD-001` rule-based fraud pre-checks (velocity / IP reuse / self-referral — pure backend logic, ties to Mukul's wallet gate later).
- **[V+M] Contract:** the `AIService` interface and the `ai_scores` write pattern (entity_type/score_type/contributing_factors/explanation). Both build against this shape.
- **Milestone:** `v0.7-synthetic-quality`

Week 9 — AI Pt 2: Fraud + Churn

- [V] `AI-FRD-002` (already trained) → `AI-FRD-003` fraud scoring service → `AI-FRD-004` wire into click flow; `AI-CHURN-002` churn service, `AI-CHURN-003` scheduled scoring trigger.
- [M] `AI-FRD-005` fraud review API, `AI-FRD-006` **withdrawal fraud gate** (his wallet code — perfect ownership fit), `AI-FRD-007` fraud admin UI; `AI-CHURN-004` churn UI.
- **[V+M] Integration seam:** fraud score (V) → fraud flag → withdrawal auto-approval block (M). Agree exactly how `WithdrawalService` queries active `pending_review` flags.
- **Milestone:** `v0.8-fraud-churn`

Week 10 — AI Pt 3: Ranking + Discovery + XAI

- [M] `AI-RANK-001/002/003` (Mukul's full AI vertical — composite scoring + ranked list + UI), `AI-DISC-002` discovery ingestion API (CSV/JSON), `AI-DISC-003` discovery UI.

- [V] `AI-DISC-001` embedding/similarity model, `AI-XAI-001` SHAP explainer, `AI-XAI-002` attach explanations to every score.
- **[V+M]** `AI-XAI-003` explanation UI — shared, because it cross-cuts every score display (quality, fraud, churn, rank, potential).
- **Integration checkpoint #3:** all seven AI features produce scores **with** explanations; AI failures do not break conversion/commission.
- **Milestone:** `v0.9-ai-complete` — **end of Phase 4**

Week 11 — Hardening + Integration testing

- [M] `TST-001` core API integration tests, `DEMO-002` Docker production config.
- [V] `TST-002` graceful degradation test (all AI methods raise → core still commits, null scores), `TST-003` performance validation (API < 2s, scoring < 1s).
- **[V+M]** `DEMO-001` demo seed script; bug-fix and refactor sweep.
- **Integration checkpoint #4:** disable the AI module entirely — referral, commission, and payout still pass.
- **Milestone:** `v1.0-rc` (release candidate)

Week 12 — Buffer + Demo prep + Final presentation

- [V+M] Final bug fixes, doc polish (update README, fill `.env.example`), rehearse the demo script (referral → click → conversion → commission → fraud alert → withdrawal → admin review), record a backup demo video, prepare slides/report.
- **Milestone:** `v1.0` (tagged release) — **end of Phase 5**

---

3. Developer Ownership

Module / Feature	**Owner**	**Why**
Docker, frontend scaffold	Mukul	Owns deployment surface he'll harden in Phase 5.
Backend scaffold, **DB models + migrations**	Vidyuth	AI work demands deep schema fluency; Vidyuth lives in the data.
Authentication + RBAC	Mukul	Security-critical, self-contained, technically meaty (JWT, bcrypt, role gates).

Module / Feature	Owner	Why
Settings	Vidyuth	Every AI threshold (fraud, churn, ranking weights) lives here — natural pairing with the AI owner.
Referral Management (links, codes, QR, clicks, conversion, attribution)	Vidyuth	Produces the exact data the quality/fraud models consume. Closes the data→model loop in one head.
Commission Management	Mukul	Money math, atomic transactions, immutability rules. Highest-stakes correctness work.
Partner Management + Tiers	Mukul	Tier-upgrade business logic + admin CRUD; pairs with commission.
Wallet & Payout	Mukul	Ledger integrity, auto-approval logic, the fraud gate. Owns all of `wallet_service.py`.
Admin dashboard + CSV	Mukul	Aggregation queries over the platform he built.
Partner dashboard	Vidyuth	Pairs with his referral/score work; gives him the partner-facing UI slice.
Synthetic data, Quality, Fraud (ML), Churn, Discovery (model), XAI	Vidyuth	Per the brief — Vidyuth owns the ML-heavy core: scikit-learn, Isolation Forest, sentence-transformers, SHAP.
Smart Partner Ranking (full vertical)	Mukul	Mukul's flagship AI feature: weighted composite is business-logic-heavy, not model-training-heavy. Reads

Module / Feature	Owner	Why
		existing scores, perfect for his strengths.
Fraud operations (rules, review API, withdrawal gate, fraud UI)	Mukul	The *operational* half of fraud. Vidyuth owns detection; Mukul owns review + enforcement, which lives in his wallet/admin code. Genuine AI-integration experience without ML training burden.
Discovery ingestion API + UI	Mukul	Vidyuth supplies `PartnerDiscovery.score()`; Mukul builds the CSV upload, endpoint, and admin page.
XAI explanation UI	Shared	Cross-cuts every score card both developers built.
Integration/degradation/perf tests, demo, prod config	Shared (split by surface)	Each tests what they own; degradation test goes to the AI owner.

Both developers demonstrably touch DB, backend, frontend, tests, and docs. Vidyuth's full-stack proof: the Referral vertical + every AI feature's own UI cards. Mukul's: four core verticals (auth, commission, partner, wallet) each spanning model→service→route→page→test, plus the admin dashboard and a complete AI feature (Ranking).

---

4. Collaboration Points

These are the moments where working alone causes pain later. Schedule them as short, deliberate joint sessions.

#	Collaboration point	When	What you agree on
1	**Database schema sign-off**	Week 1	Final tables, constraints, indexes, the `is_synthetic` discipline. Migration is locked only after both approve. Mistakes here cascade into every layer.
2	**API contracts**	Weeks 2, 3, 8	Request/response shapes for auth, settings, the conversion→commission handoff, and the `AIService`/`ai_scores` interface. Write them in `docs/` *before* the consuming side is coded.
3	**AIService + BackgroundTask wiring**	Week 8	How scoring is triggered without blocking the request, how exceptions are swallowed, how null scores are written (the NFR-REL-001 resilience contract).
4	**Fraud → withdrawal gate**	Week 9	The exact query `WithdrawalService` runs for active `pending_review` flags. Vidyuth produces flags; Mukul consumes them.
5	**XAI explanation UI**	Week 10	The shared component/tooltip rendered next to

#	Collaboration point	When	What you agree on
			every score, so both developers' score cards stay consistent.
6	**Integration + graceful-degradation testing**	Week 11	Run the full happy path together; verify the platform survives the AI module being disabled.
7	**Deployment + final demo**	Weeks 11–12	Prod Docker config, demo seed state, rehearsal, backup recording.

---

5. Git Workflow

For a disciplined two-person team, use **GitHub Flow** (protected `main` + short-lived feature branches), not full GitFlow — less ceremony, fewer stale branches.

Branching strategy

- `main` is protected and always deployable. No direct pushes.
- One feature branch per task ID. Keep branches short-lived (merge within a few days) to avoid drift.
- Optionally cut a `develop` integration branch only during Weeks 8–10 (AI integration), where many small pieces land fast; merge `develop`→`main` at each AI milestone.

Feature branch naming

<type>/<module>-<short-description> e.g.

feat/auth-jwt-issuance (AUTH-002)

feat/referral-click-tracking (REF-003)

feat/ai-fraud-isolation-forest (AI-FRD-002)

fix/wallet-negative-balance-guard

chore/docker-compose-services

docs/api-contract-conversions

test/commission-integration

Optionally append the task ID: `feat/referral-click-tracking-REF-003`.

Pull request process

- One PR per task. Keep them small (a PR you can review in 15 minutes is a good PR).
- PR description links the task ID and notes any schema/contract change.
- CI must pass: `alembic upgrade head`, `pytest`, frontend `npm run lint`/`build`.
- **At least one approval from the other developer** before merge. The author never self-merges a substantive PR.

Merge policy

- **Squash merge** to keep `main` linear and readable.
- Resolve conflicts on the feature branch by rebasing onto the latest `main`, never by merging `main` into the branch repeatedly.
- Never edit an existing Alembic migration — always add a new revision.

Code review responsibilities

- Each developer reviews the other's PRs by default.
- **Final-reviewer specialization:** Vidyuth is the required reviewer for anything touching DB schema, models, or the AI module. Mukul is the required reviewer for anything touching auth/security or the commission/wallet money path. This ensures the highest-risk surfaces always get the more-context eyes.

Commit message conventions (Conventional Commits)

<type>(<scope>): <summary> e.g.

feat(referral): add click tracking endpoint (REF-003)

fix(wallet): prevent debit below zero balance

test(commission): cover tier-based calculation

docs(api): document conversion request contract

chore(deps): pin sentence-transformers version

Types: `feat, fix, docs, test, refactor, chore`.

---

6. Development Order (and why)

Build in this exact order. The ordering rule is: **never build a consumer before its producer, and finish the data-producing spine before the intelligence that reads it.**

1. **Environment + schema** — nothing compiles or persists without these. Schema first because every later layer (and the entire ML track) imports the models.
2. **Auth + RBAC** — every protected route depends on it; building routes first means retrofitting security, which is error-prone.
3. **Settings** — commission, attribution, and AI thresholds all *read* settings at runtime. Building those before settings exists forces hardcoded values you'd later rip out (violating the config-driven principle).
4. **Referral capture (link → click → conversion → attribution)** — this is the data spine. Commission has nothing to calculate until a conversion exists.
5. **Commission → wallet credit → transaction** — triggered by a conversion. Must exist before tier upgrade (which fires on credit) and before any wallet balance is real.
6. **Partner tiers + tier upgrade** — depends on commission credits accumulating.
7. **Wallet & payout** — withdrawal needs a funded wallet; the debit path reuses the credit infrastructure from step 5.
8. **Dashboards** — pure read layer; build last among platform work so it aggregates *finished* data, not a moving target.
9. **Synthetic data** (parallel, from Week 3) — gate for all model training; produced early in `ml/` so models are ready by integration time.
10. **Quality, Fraud, Churn scoring services** — wire trained models into the platform. Built after the platform produces real features to score.
11. **Ranking** — composite of quality + churn, so it must come *after* both of those scoring services exist.
12. **Discovery** — independent; slotted here because it needs existing partner profiles to compare against.
13. **XAI** — cross-cuts all models; built once all three are trained so the explainer covers every score type.
14. **Hardening + tests + demo** — last, because integration tests and degradation tests need the full system standing.

Why this minimizes rework: each feature is built only once its inputs are stable, so you never tune a model against features that later change shape, never calculate commission against a settings format you later redesign, and never write dashboard aggregations against tables that

later get new columns. The one deliberate parallelization — the ML track in `ml/` — is safe precisely because it depends only on the *schema*, which is frozen in Week 1.

---

7. Integration Checkpoints

Checkpoint	After	Should already work	Demonstrate	Test
#1 Conversion spine	Week 4	Click → conversion → commission → wallet credit, all in one transaction	Record a conversion via API, see a `Commission` and a `Transaction` row, wallet balance increase	Commission amount correct for percentage + fixed + tier-based; transaction `balance_after` matches wallet
#2 Core platform (Phase 2 exit)	Week 6	Full referral→payout lifecycle	Partner UI: generate link, simulate click, record conversion, see commission, request withdrawal (auto-approved < threshold, queued > threshold); tier upgrade fires at threshold	Withdrawal auto-approval gate; tier upgrade is one-way; audit log on admin actions
#3 AI complete (Phase 4 exit)	Week 10	All 7 AI features produce scores **with explanations**	Fraud alert appears on admin dashboard and blocks a withdrawal's auto-approval; churn at-risk list; quality badge	Every `ai_scores` row has `contributing_factors` + `explanation`; fraud flag → withdrawal gate; scores excluded

Checkpoint	After	Should already work	Demonstrate	Test
			with SHAP tooltip; ranked partner list; discovery candidate scores	when `is_synthetic`
#4 Hardening (Phase 5 exit)	Week 11	Whole system stable under the demo flow	Disable AI module entirely — referral, commission, payout still pass	Graceful degradation (AI raises → null score, core commits); API < 2s, scoring < 1s; integration suite green

---

8. Risk Analysis

High-risk modules

- **Fraud detection (`AI-FRD-002/003`)** — Isolation Forest tuning; false positives block real payouts. *Mitigation:* train and evaluate in Week 6 (well before integration), tune the `fraud.alert_threshold` against synthetic precision/recall, keep the rule-based path independent so detection survives a bad model (already in the architecture).
- **Partner Discovery (`AI-DISC-001`)** — `sentence-transformers` is a heavy dependency (model download, image bloat, embedding latency). *Mitigation:* pin the model (`all-MiniLM-L6-v2`), cache it in a dedicated Docker layer, and treat Discovery as the **first feature to descope** if time runs short (it has no downstream dependents).
- **Commission + Wallet atomicity** — money correctness; a partial commit corrupts the ledger. *Mitigation:* single DB transaction for calc + credit + transaction row; Mukul owns the whole path; integration tests assert balance reconstruction from the transactions table.
- **XAI (`AI-XAI-001`)** — SHAP behaves differently across model types and can be slow. *Mitigation:* `TreeExplainer` only (all three models are tree-based by design), precompute top-3 factors, fall back to feature-importance templates if SHAP misbehaves.
- **Async SQLAlchemy 2.0 in BackgroundTasks** — session lifecycle in background scoring is a classic source of bugs. *Mitigation:* a dedicated session per background task, never reuse the request session; test in Week 8 when the first scoring task lands.

Potential merge conflicts (hot files)

- `app/main.py` (router registration), `app/models/__init__.py`, `app/api/v1/routes/__init__.py`, `requirements.txt`, frontend `AppRouter.jsx` and `Sidebar.jsx`, and **Alembic `versions/`** (two devs auto-generating migrations the same week is the #1 conflict risk).
- *Mitigation:* designate a **single migration owner per week** (the person doing schema-affecting work); always `git pull --rebase` before `alembic revision --autogenerate`; treat router/registration edits as tiny standalone PRs that merge fast; one component file per module in the frontend API layer (already the structure).

Technical bottlenecks

- Synthetic data quality is the ceiling on every model — garbage in, garbage out. *Mitigation:* invest in realistic distributions and labels in Week 3–5; eyeball the data in `01_synthetic_data_exploration.ipynb` before trusting any model.
- Docker image size from ML dependencies. *Mitigation:* multi-stage build, cache the transformer model, keep ML deps in the backend image only if needed at runtime.

AI-related risks

- Models miss SRS eval targets (quality ROC-AUC ≥ 0.75, churn PR-AUC). *Mitigation:* gate this in Week 7 — if a model misses, fix data/features then, not in Week 10.
- Overfitting to synthetic patterns. *Mitigation:* held-out synthetic test set, report metrics in `ml/evaluation/`.

Time-management risks

- Internal assessments / exams compress mid-semester. *Mitigation:* Week 7 is pre-planned as low-velocity; the parallel ML track front-loads the heaviest work.
- Phase 4 (82h) is the most underestimated phase. *Mitigation:* the ML track makes Weeks 8–10 integration, not invention; plus a **descope ladder** (below).
- Scope creep against the BRD. *Mitigation:* the BRD §4.2 / SRS §10 "not built" lists are binding — if it's not in scope, it's not in this semester.

Descope ladder (cut in this order if behind): Partner Discovery UI polish → Discovery entirely → CSV export → Ranking UI niceties → reduce XAI to feature-importance (skip SHAP). Never descope: the referral→commission→payout spine, fraud detection, or graceful degradation — those are the success-criteria core.

---

9. Buffer Weeks

Buffer is reserved explicitly, not borrowed from feature time:

- **Week 7** — low-velocity slack absorbing internal assessments; dashboard work only, no critical-path AI.
- **Week 11** — dedicated hardening: bug fixing, refactoring, integration + degradation + performance testing, prod Docker config.
- **Week 12** — documentation polish, demo rehearsal, backup recording, presentation/report prep.
- **Distributed slack** — Week 6 carries Phase 2 catch-up; the parallel ML track means an AI slip doesn't stall the platform and vice-versa.

If everything runs on time, Weeks 11–12 become the difference between a working project and a *polished, presentable* one — which is what a capstone is graded on.

---

10. Summary — One-Page View

- **Shape:** Foundation (W1–2) → Core platform (W3–6) → Dashboards (W7) → AI integration (W8–10) → Hardening + demo (W11–12), with a **parallel ML R&D track running W3–7 inside `ml/`.**
- **Split:** Vidyuth owns the data spine (referral, settings) + the ML module + DB schema. Mukul owns the money/identity spine (auth, commission, wallet, payouts, partners) + admin dashboard + the Ranking AI feature + fraud operations. Both ship full vertical slices; both touch DB, backend, frontend, tests, docs.
- **Conflict control:** vertical ownership, folder isolation (`ml/` vs `backend/`), single-owner hot files (`wallet_service.py`, weekly migration owner), small squash-merged PRs with mandatory cross-review.
- **Risk control:** ML front-loaded and eval-gated by Week 7; resilience (graceful degradation) is a tested success criterion, not an afterthought; a written descope ladder protects the core if time runs short.
- **Source of truth stays the BRD/SRS.** This plan only governs sequencing and ownership; it never adds scope beyond those documents.